

First steps

- Goal: to simulate subsystem codes and compare their logical error rates to current standards (surface codes, etc.)
- Understand subsystem codes and how they allow functional advantage
- Learn to use Stim, work with Stim's detector logic
- Implement and play around with some basic codes: Shor, rotated surface code, Bacon-shor

- Basic design for first logical error rate comparison experiment:
 - Run both Shor and Bacon-shor under parameter scan:
 - Physical error probabilities: [1e-5, 2e-5, 5e-5, 1e-4, 2e-4, 5e-4, 1e-3, 2e-3, 5e-3, 1e-2]
 - Noise round lengths: [1, 2, 5, 10, 20, 50, 100, 200, 500, 1000]
 - Total of 10x10=100 measured LERs per circuit, plot against single qubit behavior.

First steps - Shor code, stim code and layout

Stim code I implemented for shor code, errors applied, all the same for now, this shows the last iteration of parameter scan.

```
stim.Circuit('''
R 0 1 2 3 4 5 6 7 8
X ERROR(0.01) 0 1 2 3 4 5 6 7 8
H 0 3 6
DEPOLARIZE1(0.01) 0 3 6
CX 0 1 3 4 6 7
DEPOLARIZE2(0.01) 0 1 3 4 6 7
CX 1 2 4 5 7 8
DEPOLARIZE2(0.01) 1 2 4 5 7 8
R 9 10 11 12 13 14 15 16 17 18 19 20 21 22
X ERROR(0.01) 9 10 11 12 13 14 15 16 17 18 19 20 21 22
CX 0 9 1 10
DEPOLARIZE2(0.01) 0 9 1 10
CX 1 9 2 10
DEPOLARIZE2(0.01) 1 9 2 10
MR(0.01) 9 10
X ERROR(0.01) 9 10
CX 3 9 4 10
DEPOLARIZE2(0.01) 3 9 4 10
CX 4 9 5 10
DEPOLARIZE2(0.01) 4 9 5 10
MR(0.01) 9 10
X ERROR(0.01) 9 10
CX 6 9 7 10
DEPOLARIZE2(0.01) 6 9 7 10
CX 7 9 8 10
DEPOLARIZE2(0.01) 7 9 8 10
MR(0.01) 9 10
X ERROR(0.01) 9 10
H 11 17
DEPOLARIZE1(0.01) 11 17
CX 11 12 17 18
DEPOLARIZE2(0.01) 11 12 17 18
CX 11 13 17 19
DEPOLARIZE2(0.01) 11 13 17 19
CX 11 14 17 20
DEPOLARIZE2(0.01) 11 14 17 20
CX 11 15 17 21
DEPOLARIZE2(0.01) 11 15 17 21
CX 11 16 17 22
DEPOLARIZE2(0.01) 11 16 17 22
CX 11 0 12 1 13 2 14 3 15 4 16 5
DEPOLARIZE2(0.01) 11 0 12 1 13 2 14 3 15 4 16 5
CX 17 3 18 4 19 5 20 6 21 7 22 8
DEPOLARIZE2(0.01) 17 3 18 4 19 5 20 6 21 7 22 8
H 11 12 13 14 15 16 17 18 19 20 21 22
DEPOLARIZE1(0.01) 11 12 13 14 15 16 17 18 19 20 21 22
MR(0.01) 11 12 13 14 15 16 17 18 19 20 21 22
X ERROR(0.01) 11 12 13 14 15 16 17 18 19 20 21 22
)
MX 0 1 2 3 4 5 6 7 8
OBSERVABLE INCLUDE(0) rec[-9] rec[-8] rec[-7] rec[-6] rec[-5]
rec[-4] rec[-3] rec[-2] rec[-1]
''')
```

```
REPEAT 1000 {
  DEPOLARIZE1(0.01) 0 1 2 3 4 5 6 7 8 (Idle noise)
  CX 0 9 1 10
  DEPOLARIZE2(0.01) 0 9 1 10
  CX 1 9 2 10
  DEPOLARIZE2(0.01) 1 9 2 10
  MR(0.01) 9 10
  X ERROR(0.01) 9 10
  CX 3 9 4 10
  DEPOLARIZE2(0.01) 3 9 4 10
  CX 4 9 5 10
  DEPOLARIZE2(0.01) 4 9 5 10
  MR(0.01) 9 10
  X ERROR(0.01) 9 10
  CX 6 9 7 10
  DEPOLARIZE2(0.01) 6 9 7 10
  CX 7 9 8 10
  DEPOLARIZE2(0.01) 7 9 8 10
  MR(0.01) 9 10
  X ERROR(0.01) 9 10
  H 11 17
  DEPOLARIZE1(0.01) 11 17
  CX 11 12 17 18
  DEPOLARIZE2(0.01) 11 12 17 18
  CX 11 13 17 19
  DEPOLARIZE2(0.01) 11 13 17 19
  CX 11 14 17 20
  DEPOLARIZE2(0.01) 11 14 17 20
  CX 11 15 17 21
  DEPOLARIZE2(0.01) 11 15 17 21
  CX 11 16 17 22
  DEPOLARIZE2(0.01) 11 16 17 22
  CX 11 0 12 1 13 2 14 3 15 4 16 5
  DEPOLARIZE2(0.01) 11 0 12 1 13 2 14 3 15 4 16 5
  CX 17 3 18 4 19 5 20 6 21 7 22 8
  DEPOLARIZE2(0.01) 17 3 18 4 19 5 20 6 21 7 22 8
  H 11 12 13 14 15 16 17 18 19 20 21 22
  DEPOLARIZE1(0.01) 11 12 13 14 15 16 17 18 19 20 21 22
  MR(0.01) 11 12 13 14 15 16 17 18 19 20 21 22
  X ERROR(0.01) 11 12 13 14 15 16 17 18 19 20 21 22
  )
  DETECTOR rec[-18] rec[-36]
  DETECTOR rec[-17] rec[-35]
  DETECTOR rec[-16] rec[-34]
  DETECTOR rec[-15] rec[-33]
  DETECTOR rec[-14] rec[-32]
  DETECTOR rec[-13] rec[-31]
  DETECTOR rec[-12] rec[-11] rec[-10] rec[-9] rec[-8] rec[-7]
  rec[-30] rec[-29] rec[-28] rec[-27] rec[-26] rec[-25]
  DETECTOR rec[-6] rec[-5] rec[-4] rec[-3] rec[-2] rec[-1]
  rec[-24] rec[-23] rec[-22] rec[-21] rec[-20] rec[-19]
  )
  MX 0 1 2 3 4 5 6 7 8
  OBSERVABLE INCLUDE(0) rec[-9] rec[-8] rec[-7] rec[-6] rec[-5]
  rec[-4] rec[-3] rec[-2] rec[-1]
  )
}
```

Color coded parts:

Data qubit preparation

Ancilla qubit preparation

Initial Z checks (no detector)

Initial X checks (no detector)

Repeat Z checks

Repeat X checks

Z Detector with current round and previous round of measurements

X Detector with current and previous rounds

Final readout

Circuit and detector_error_model decoded with pymatching

*X errors after MR and R represent reset errors

First steps - Bacon shor code, stim code and layout

Stim code I implemented for Bacon shor, errors applied, all the same for now, last iteration of parameter scan.

```
stim.Circuit('''
  R 0 1 2 3 4 5 6 7 8
  X ERROR(0.01) 0 1 2 3 4 5 6 7 8
  R 9 10 11 12 13 14 15 16 17 18 19 20
  X ERROR(0.01) 9 10 11 12 13 14 15 16 17 18 19 20
  CX 0 9 1 10 3 11 4 12 6 13 7 14
  DEPOLARIZE2(0.01) 0 9 1 10 3 11 4 12 6 13 7 14
  CX 1 9 2 10 4 11 5 12 7 13 8 14
  DEPOLARIZE2(0.01) 1 9 2 10 4 11 5 12 7 13 8 14
  MR(0.01) 9 10 11 12 13 14
  X ERROR(0.01) 9 10 11 12 13 14
  H 15 16 17 18 19 20
  DEPOLARIZE1(0.01) 15 16 17 18 19 20
  CX 15 0 16 3 17 1 18 4 19 2 20 5
  DEPOLARIZE2(0.01) 15 0 16 3 17 1 18 4 19 2 20 5
  CX 15 3 16 6 17 4 18 7 19 5 20 8
  DEPOLARIZE2(0.01) 15 3 16 6 17 4 18 7 19 5 20 8
  H 15 16 17 18 19 20
  DEPOLARIZE1(0.01) 15 16 17 18 19 20
  MR(0.01) 15 16 17 18 19 20
  X ERROR(0.01) 15 16 17 18 19 20
  REPEAT 1000 {
    TICK
    DEPOLARIZE1(0.01) 0 1 2 3 4 5 6 7 8 (Idle noise)
    CX 0 9 1 10 3 11 4 12 6 13 7 14
    DEPOLARIZE2(0.01) 0 9 1 10 3 11 4 12 6 13 7 14
    CX 1 9 2 10 4 11 5 12 7 13 8 14
    DEPOLARIZE2(0.01) 1 9 2 10 4 11 5 12 7 13 8 14
    MR(0.01) 9 10 11 12 13 14
    X ERROR(0.01) 9 10 11 12 13 14
    H 15 16 17 18 19 20
    DEPOLARIZE1(0.01) 15 16 17 18 19 20
    CX 15 0 16 3 17 1 18 4 19 2 20 5
    DEPOLARIZE2(0.01) 15 0 16 3 17 1 18 4 19 2 20 5
    CX 15 3 16 6 17 4 18 7 19 5 20 8
    DEPOLARIZE2(0.01) 15 3 16 6 17 4 18 7 19 5 20 8
    H 15 16 17 18 19 20
    DEPOLARIZE1(0.01) 15 16 17 18 19 20
    MR(0.01) 15 16 17 18 19 20
    X ERROR(0.01) 15 16 17 18 19 20
    DETECTOR rec[-12] rec[-10] rec[-8] rec[-24] rec[-22] rec[-20]
    DETECTOR rec[-11] rec[-9] rec[-7] rec[-23] rec[-21] rec[-19]
    DETECTOR rec[-6] rec[-4] rec[-2] rec[-18] rec[-16] rec[-14]
    DETECTOR rec[-5] rec[-3] rec[-1] rec[-17] rec[-15] rec[-13]
  }
  M 0 3 6
  OBSERVABLE_INCLUDE(0) rec[-3] rec[-2] rec[-1]
''')
```

Color coded parts:

Data qubit preparation
Ancilla qubit preparation
Initial Z checks (no detector)
Initial X checks (no detector)
Repeat Z checks
Repeat X checks
Z Detector with current round and
previous round of measurements
X Detector with current and
previous rounds
Final readout

Circuit and detector_error_model
decoded with pymatching

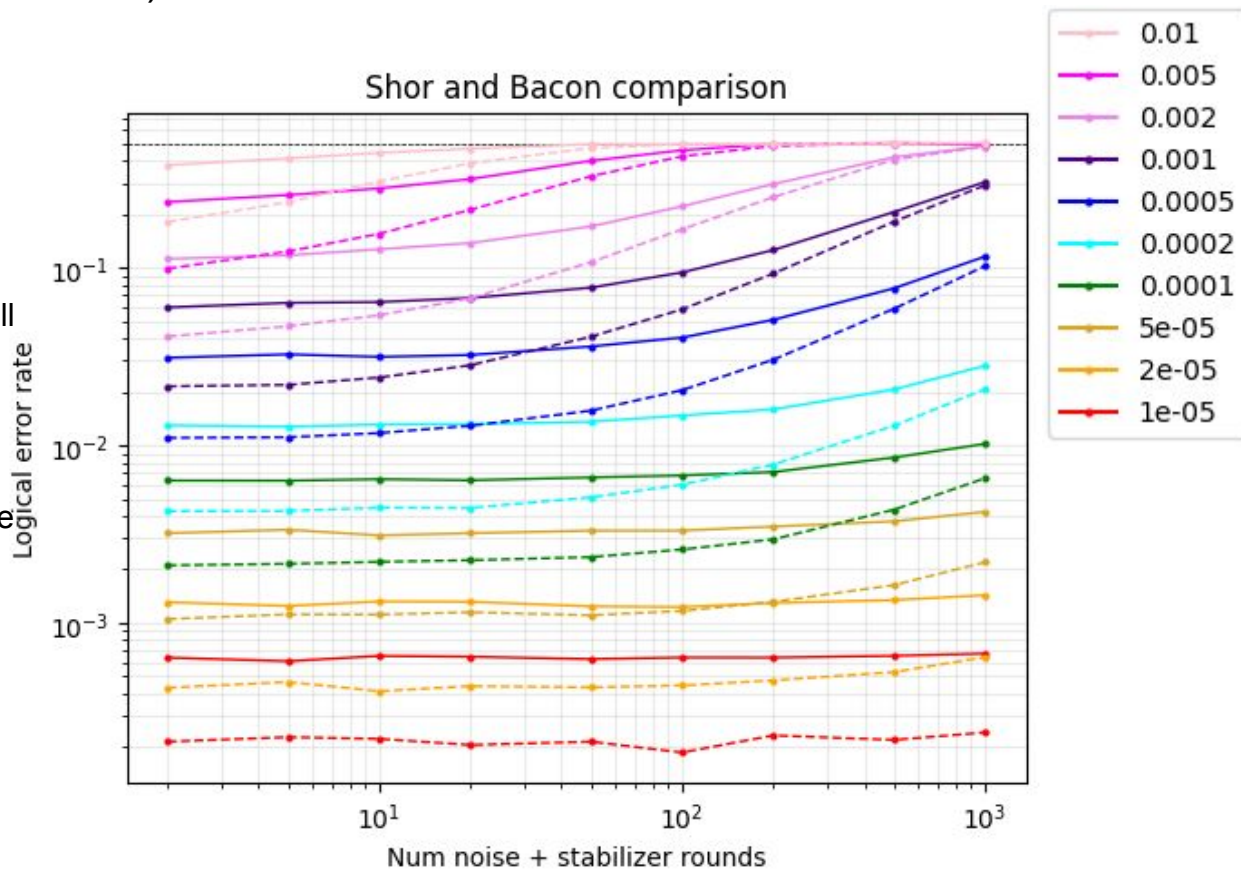
*X errors after MR and R
represent reset errors

First steps - Plots (x axis num rounds)

Logical error rate comparison between Shor (solid) and Bacon shor (dashed) for each noise level plotted over many rounds. Observed trends:

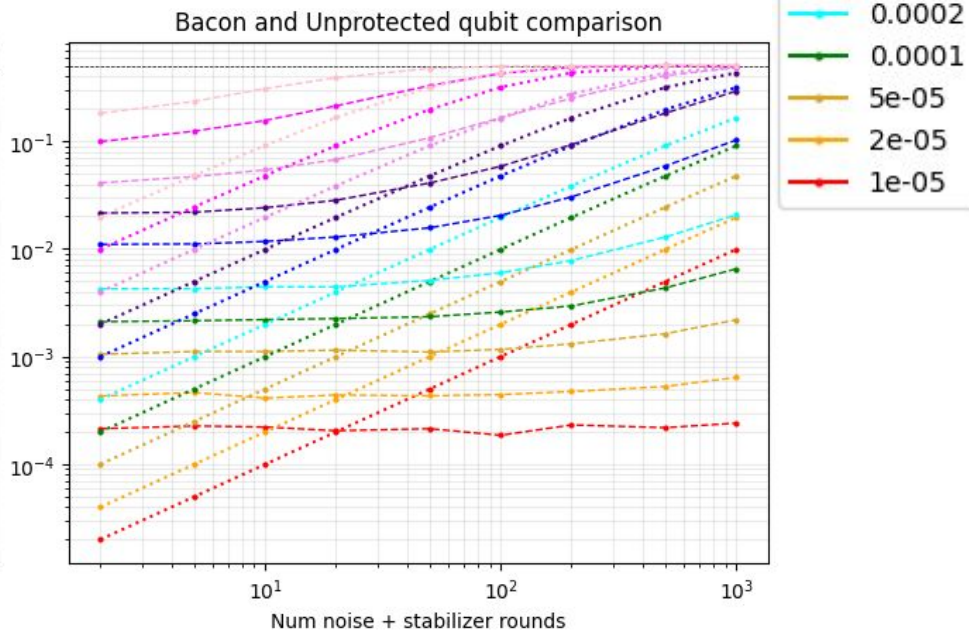
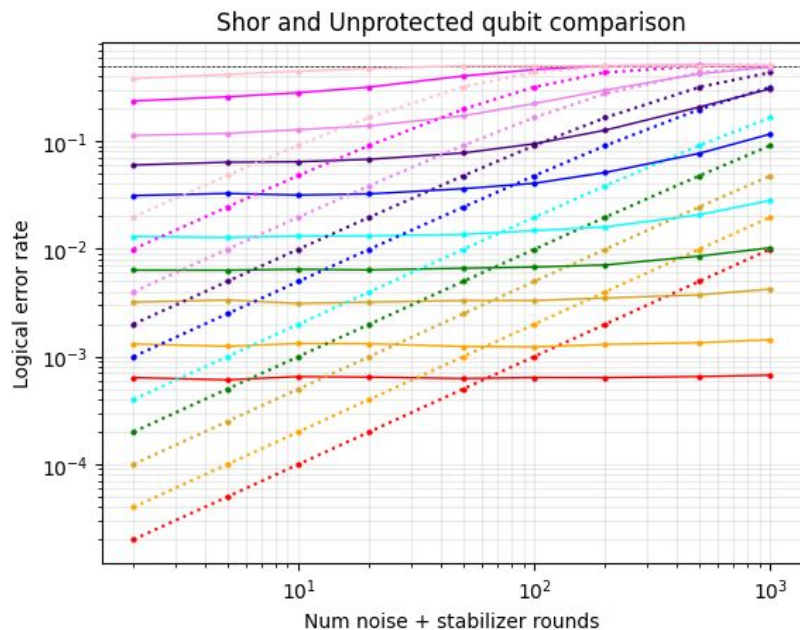
- Bacon shor performs better for all noise levels at lower rounds, but converges onto Shor at higher rounds.
- Starting LER at noise round 1 is $\sim 100\times$ the noise level for all noise levels, not sure if this makes sense? From modeling noisy circuit measurements possibly?

(Colors don't have anything to do with previous slides)



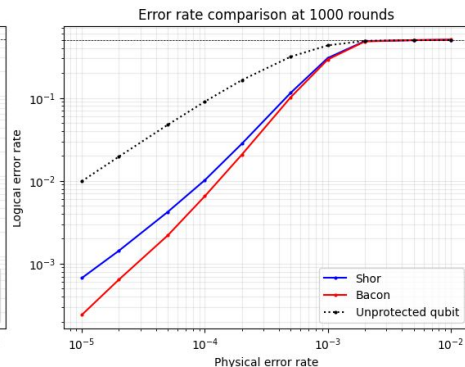
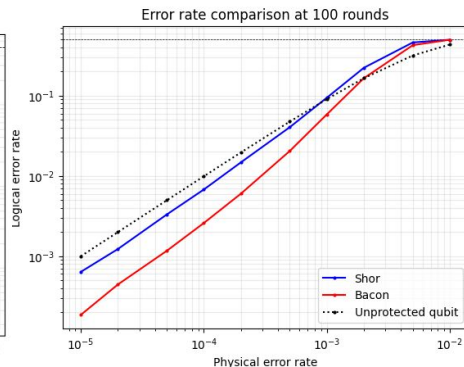
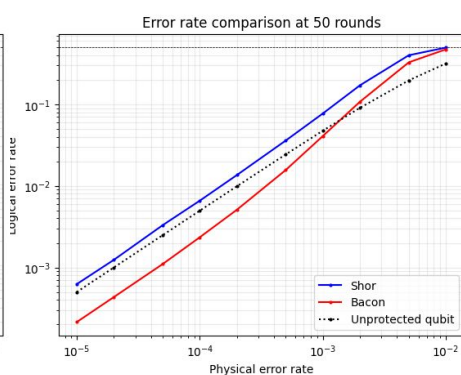
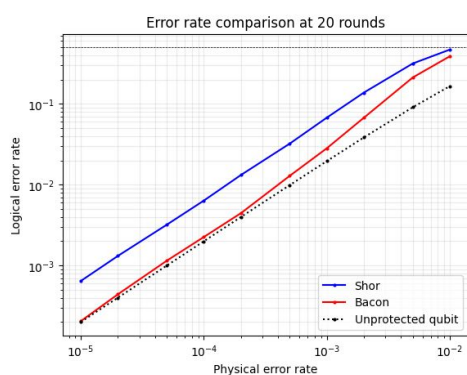
First steps - Plots (x axis num rounds)

Comparison to unprotected qubit (dotted) that flips with probability in legend, we see good performance for larger noise rounds. Shor and Bacon plotted separately so its hopefully not too cluttered.



First steps - Plots (x axis physical error rate)

Some plots at different noise rounds showing LER against physical error rate. These essentially completely overlap on the same plot so I've plotted noise rounds individually. We see better performance than unprotected qubit for large number of rounds for sufficiently small physical error rate. Also, Bacon performs consistently better than Shor, ~3x smaller LER for most physical error rates sampled.



Next steps

- Move away from simple noise model, look to model noise at more realistic levels, test varying noise types against each other.
- Compare surface codes and the gross code (still reading on how the gross code works exactly)